Benchmark tuning round --- consolidated findings

detector-opt, state as of 2026-08-13

2026-08-13

Contents

State as of 2026-08-13 21:00. This CONSOLIDATES; it supersedes nothing. `docs/benchmark-acceptance.md` is the binding criterion, `docs/tuning-preregistration.md` governs this round, `docs/decision-log.md` is the narrative. Where this document and the pre-registration disagree, the pre-registration wins.

Goal (user): a realistic task, or tasks, suitable for demonstrating `meta`'s ~2x iteration advantage — tuned, and verified on the actual neural campaign.

Units. Every effect is given in absolute loss AND as a multiple of the config `loss_precision` (8.0e-3 for `extremes`). Standard errors over held-out EVENTS are NOT used as the yardstick anywhere here: they answer "how precisely did I measure this one network", whereas criterion (d) is stated in `loss_precision` and an effect that cannot move that bar cannot matter. Numbers repeated from older notes that were originally quoted in event-sigma are re-stated here in `loss_precision` units.

! D53's OTHER justification for that choice is now REFUTED and must not be repeated. It claimed the same condition drifts 0.003–0.009 between nominally identical runs. `pn-s7-old` re-ran the stage-1 growth arm at byte-identical settings and seeds and reproduced it to **0.0002 (0.02 x `loss_precision`)**, with epoch and round counts identical to the step (1.00x on all three seeds). The trainer is deterministic given its seed up to GPU reduction order; the 0.003–0.009 was across-SEED or across-SETTING variation misattributed to re-running. **The replication floor is 0.0002, and every paired difference in this document should be read against it.**

One thing that floor does NOT explain, and it is now an open question: four nominally identical runs of `scripts/probe_meta_cripple.py` at the same seed gave arm C held-out losses spanning 0.0094 (§3.1). If the trainer replicates to 0.0002, those four runs differed in something that was not recorded.

Censoring. A cell that reports "converged" stopped on the FIRST epoch under its bar: it proves `diff + err <= bar`, not what its floor is. Only a CAPPED cell (exhausted `iteration_limit`) reports an uncensored slack. Nothing below differences a converged cell against a capped one.

---

1. The verdict on the two candidate tasks

1.1 `mm-sym` — REJECTED ON THE PROXY, and that rejection is now UNDER TEST

One cell returned: `mmsym-m4-r16` (m = 4, 16 read-outs, 512 Sobol designs, 16384 events, 10 seeds). Source `output/screen/mmsym-m4-r16.log` (verdict block at the end; the job was cancelled after printing it, so no `.json` was written). The other two `mm-sym` cells were cancelled before finishing.

```
landscape   ceiling 0.3332   best 0.1406   baseline 0.2438   worst 0.3335   span 0.1032
            at-ceiling 4.7%  top-decile 5.8%  GP R2 +0.856  8.27 s/design

20 -> 40    (a) 10/10   (b) 7/10   (c) 3/10   (d) 0/10
            STRONG 0/10 (need > 5)    WEAK 7/10 (need 10)    median gain 0.0125

BO @20 median 0.1205   BO @40 median 0.1078   (BO beats the sampled best: it optimises
                                              continuously, not over the 512 Sobol points)
```

The proxy rejection does not depend on `loss_precision`, which for `mm-sym` is UNMEASURED and was run at a declared placeholder. Conditions (b) and (c) carry no bar: three seeds gained exactly 0.0000 from 20 to 40 designs, so WEAK fails outright, and only 3 of 10 seeds cleared the bar-free (c). The bar arithmetic is also hostile: `10 x 0.01` = 0.100 against a span of 0.1032, i.e. 97% of the whole baseline-to-best range, which pre-registration §5.1 calls an automatic reject.

! BUT THE PROXY IS THE SUSPECT. The user's objection, and it is the strongest argument in this document: `mm-sym` estimates THREE coefficients (`log_K_A`, `log_K_B`, `Q10_cat`) from clean progress curves, so a couple of non-degenerate noiseless experiments should pin them almost exactly and the information floor should be near ZERO. A "best design" at 0.1406 against a no-information level of 1/3 is an ESTIMATOR floor, not a task floor. XGBoost sees the read-outs FLATTENED — `m * n_measurements` columns, 64 at this cell — and deliberately WITHOUT the design coordinates (`detopt/bo/gbdt.py`, `sample_design`), while the neural objective sees one set element per experiment carrying its own read-outs AND its own design coordinates and is permutation-invariant across experiments (`combined_event_shape`, `detopt/detector/enzyme_mm.py:433`).

Two jobs are settling it, both CPU-only so they do not compete for the shards:

```
593    mm-neural              the proxy's BEST / MEDIAN / WORST designs re-
scored through the SetRegressor
                at m=4, 16 read-outs, 2 seeds  -> output/screen/mmsym-neural-bmw.json
600  mm-neural-m32  m=32, designs named by CONSTRUCTION (centre, low corner, Sobol points) -
-
              there is no landscape at that batch size and deliberately none, since the
                 proxy would face 32*16 = 512 flat columns  -> mmsym-neural-m32.json
```

If the neural floor lands near zero the span becomes ~0.24 rather than 0.103, criterion (d) becomes reachable, and `mm-sym`'s rejection was a proxy artefact. **Until those return, the rejection above stands only as "rejected by the XGBoost proxy", not as "rejected".**

1.2 `extremes` — the campaign task, at m = 4, `n_measurements` = 32

Three cells complete, all at m = 4, all 512 Sobol designs x 16384 events, 10 seeds, one run to 40 with all three doubling pairs read off the same curves. Sources `output/screen/extremes-m4-r{8,16,32}.json`.

cell	best	baseline	span	dead %	top-decile	GP R2	s/design
r8 (160 steps/read)	0.6193	0.8648	0.2455	12.1	7.81%	+0.662	6.31
r16 (80)	0.5226	0.8239	0.3013	8.0	8.28%	+0.635	11.48
r32 (40)	**0.4090**	**0.7749**	**0.3659**	**5.7**	**7.75%**	**+0.594**	**11.18**

More read-outs widen the baseline-to-best span monotonically — the physics stated BEFORE it was measured. Cost is nearly flat (6.31 → 11.18 s/design, not 4x) because the TOTAL integration steps were held at 1280 across the sweep.

Criterion (1), per doubling pair, at the configured `loss_precision` 8.0e-3:

cell	pair	median gain (SEM)	(a)	(b)	(c)	(d)	STRONG	WEAK	verdict
r8	5→10	+0.0434 (0.0098)	10	9	8	2	2	9/10	fail
r8	10→20	+0.0811 (0.0161)	10	10	9	5	5	10/10	fail by one seed
r8	20→40	+0.0868 (0.0271)	10	8	7	6	6	8/10	fail (WEAK)
r16	10→20	+0.0624 (0.0171)	10	8	7	4	4	8/10	fail (WEAK)
r16	20→40	+0.0941 (0.0195)	10	9	8	7	7	9/10	fail (WEAK)
r32	5→10	+0.0653 (0.0233)	10	10	8	5	5	10/10	fail by one seed
r32	10→20	+0.0322 (0.0192)	10	9	4	3	3	9/10	fail
r32	**20→40**	**+0.1584 (0.0172)**	**10**	**10**	**9**	**9**	**9**	**10/10**	**PASS**

The 20 → 40 gain EXCEEDS the 10 → 20 gain at every cell, by 5x at r32. A saturating task does the opposite. That is the "still descending late" property the whole goal rests on, and it is why r32 is the cell to campaign at: `meta` at ~2.45x the designs of `from_scratch` puts the campaign's own comparison at 20-vs-40-plus design counts, which is exactly the pair r32 passes.

Which doubling pair counts. The user's ruling: *"Criterion is for n >= 10, doubling pays. Could be relaxed to n >= 5"* and *"10 -> 20 is enough, the higher the n the better (unless n >= 50, just too long to compute)"*. 20 → 40 is therefore an ADMISSIBLE and indeed preferred pair, not a substitution. What must still be said plainly:

1. **r32 FAILS at 10 → 20, and it fails on (c), which carries no bar.** Only 4 of 10 seeds gained more than `0.2 x (baseline - loss@n)` there. No reading of criterion (d) rescues that. r32's curve is flat-ish over 10 → 20 and descends hard from 20 to 40 — a late-descending landscape, which is what the goal wants but not what the smaller pair measures.
2. **At the configured 8.0e-3, no cell passes at 10 → 20.** r8 misses by exactly one seed on (d) — 5/10 where >5 is required — and r16 and r32 fail on (c) or WEAK. At a `loss_precision` the fixed-window schedule demonstrably delivers (§2), r8's 10 → 20 becomes STRONG 7/10, WEAK 10/10 — a PASS at bar 0.0060–0.0062 — while r32's 20 → 40 remains a PASS at every bar down to the growth-path maximum. The two pairs therefore point at DIFFERENT cells, and choosing between them is a judgement about which comparison the campaign actually runs, not a measurement. Recorded so it is made deliberately.

BONUS (3), BO vs random, paired by seed: BO wins 10/10 at every cell. At r32, BO median final 0.2880 against random 0.5063.

The neural span is 0.716x the proxy's (`output/screen/precision-arms-handover.md`): neural best 0.6372 (`sobol-rank3`), baseline 0.8991 (`sobol-median`), span **0.2619**, the two seeds agreeing (0.2578 / 0.2660). The criterion is evaluated on the neural objective, so every bar FRACTION is 1.4x larger there, and every proxy gain scales by 0.716: the r32 20 → 40 median gain of 0.1584 reads **0.1134** on the neural scale. Proxy-to-neural rank agreement over 12

Sobol designs is Spearman +0.818 — a fair but imperfect stand-in; the proxy's best design is NOT the neural best.

m = 6 has a landscape and no verdict. `extremes-m6-r32` reported best 0.3093, at-ceiling 1.0%, top-decile 5.1%, GP R2 +0.596, 24.75 s/design, then was cancelled BY THE USER at 19:43 partway through its iteration test, on the grounds that XGBoost on 6 experiments x 32 read-outs (192 flat columns) was not going to converge in reasonable time. Its dead volume of 1.0% would in any case have failed pre-registration condition (iv), which §5 item 6 flags as possibly mis-specified.

1.3 What the XGBoost proxy actually sees

`measurements` is `(n_experiments, n_measurements)` — one scalar read-out per sample — so the flat width is exactly `m * r`, design columns excluded by construction:

```
m      r=8     r=16     r=32
2       16       32       64
3       24       48       96
4       32       64      128      <- the passing extremes cell is r32, i.e. 128
6       48       96      192      <- cancelled
```

At 16384 events per design and a 0.25 validation fraction that is 12288 training rows: 192 rows per column at m4-r16, 96 at m4-r32, 64 at m6-r32. Worse than the raw count suggests, because the columns are a TIME SERIES — successive samples of one progress curve, strongly correlated, and axis-aligned splits are a poor way to read a smooth curve — and because flattening discards the experiment exchangeability the set regressor exploits.

The direction of that bias is favourable for `extremes` and hostile for `mm-sym`. For `extremes` the criterion IMPROVED monotonically as the proxy's problem got harder (STRONG 6/10 → 7/10 → 9/10 across r8/r16/r32), so the proxy understates that cell rather than inflating it. For `mm-sym`, whose whole rejection rests on a compressed span, the same handicap is the leading explanation — §1.1.

---

2. What limits the achievable precision

The trainer must drive `diff + err` below `loss_precision` before it returns an objective, and `bo.py` raises if a design cannot (`detopt/nn/trainer/design.py`). `err` is statistical and falls as `1/sqrt(window)`; `diff` is the train/validation gap and is the binding term.

2.1 The growth schedule manufactures most of the gap — measured, paired

The mechanism, visible in the round trace. `output/screen/pn-rounds.log`: every round of the growth path lasts EXACTLY 17 epochs = `warmup_epochs` (16) + 1, for 127 rounds and 2159 epochs. Rule 1 of the convergence procedure fires at the first legal epoch of every round, so the window grows before the network has ever trained out on the one it has.

It is NOT accumulating memorisation of early events: within a design the gap FALLS with the window (`sobol-median`/seed 1: 0.0864 at window 2048 → 0.0071 at 131072), and the mean gap over the first half of the rounds is 0.0269 against 0.0087 over the second half. The gap is a chronic transient, not a growing bias.

Stage 1 of the paired ablation (D55). Both arms from the SAME initial network per seed (`init |params|_1` = 545.636748 / 552.446201 / 560.388490 appear cell-for-cell in both arms), `n_models = 1`, dropout 0.1 in both, 3 designs x 3 seeds, `extremes` m=4/r32. Sources `output/screen/pn-s1-{grow,fixed}.json`.

```
grow arm:  6 of 9 cells CAPPED at the 131072 cap, still 1.4-2.7 x loss_precision above the bar
fixed arm:  9 of 9 converged, slack 0.00365 - 0.00876

fixed - grow, over the 6 pairs where the grow side capped (uncensored on that side),
per seed and then across-seed:
```

```
quantity     s1        s7      s126382657     mean      sd     x loss_precision
diff+err  -0.01290  -0.01082   -0.00735    -0.01035  0.00280      -1.29
diff      -0.01304  -0.01071   -0.00745    -0.01040  0.00281      -1.30
err       +0.00014  -0.00010   +0.00010    +0.00005  0.00013      +0.01   (null)
level     +0.00335  -0.00080   -0.00109    +0.00049  0.00249      +0.06   (null)

epochs   0.02x     rounds 0.01x     wall clock 0.89x
```

Every seed is negative and the mean is 3.7x the across-seed spread. The effect is ENTIRELY `diff`; `err` is nil. **The level is unchanged**, which is the control that mattered — the fixed window is not buying its smaller gap by underfitting.

! D55's headline of "0.0066 (0.83 x `loss_precision`)" is the mean over ALL 9 pairs including the 3 where the grow side had converged (i.e. was censored at its bar). On the 6 uncensored pairs the effect is **0.0104 = 1.30 x `loss_precision`**. Use the uncensored figure.

Three consequences the campaign needs:

- **The reported objective does not move.** Level change +0.0005 (0.06x precision). Switching schedule does not shift the landscape's level.
- **The span between designs WIDENS slightly.** Good-design (`sobol-rank6`) to bad-design (`corner-high`) span per seed: grow 0.2009 / 0.1963 / 0.2076 (mean 0.2016) against fixed 0.2134 / 0.2143 / 0.2087 (mean 0.2121) — +0.0105, and more consistent across seeds. Three designs, so suggestive, not established.
- **The penalty is DESIGN-DEPENDENT.** grow/fixed slack ratio is 1.0–1.4x at `sobol-rank6` (level ~0.71, a good design), 1.3–2.6x at `sobol-median` (~0.89), and 4.0–5.1x at `corner-high` (~0.92, a bad one). Growth inflates the reported loss of BAD designs most, which exaggerates the landscape rather than flattening it. What it breaks is the guarantee that a returned number carries the stated precision.

! CONFOUND, not yet removed. The fixed arm was asked for precision 0.03 (with `flatness_tol` compensating exactly, so it exits on the plateau test and its numbers are floors, not bar-stopped values) and therefore stopped after 17–40 epochs. The two arms differ in schedule AND in training length. Stage 6 (`pn-s6-n0`, running) removes it: `n0` in {8192, 32768} at precision 8.0e-3, everything else identical to the grow arm.

2.2 Levers that were tested and are NULL

lever	effect on the gap	evidence	strength
more data	none by construction	`err` ~ `1/sqrt(window)`, `diff` is a bias floor; reaching 0.008 by data alone needs 1.18–2.06M events on ONE design, 56–98% of the whole campaign budget	arithmetic
doubled schedule (`n0` 4096, `n_increment` 2048, warmup 32, patience 16)	−0.00009 mean (−0.01 x precision), sign flips 6/11	11 capped-in-both cells; calls/design 146767 vs 142850 (0.973x) — a null on cost too	11 paired cells, unpaired init
gap-stability gate (`require_stable_gap`)	−3.0% on `diff` (0.00786 vs 0.00810), +56% wall clock	ONE capped-in-both cell	1 cell
parameter noise eps = 0.01	**+0.00020 mean (+0.03 x precision)**, 3/5 negative	5 capped-vs-capped pairs, SAME init, `pn-s5-e01.json` vs `pn-s1-grow.json`	5 paired cells

lever	effect on the gap	evidence	strength
parameter noise eps = 0.003	**+0.00001 mean (0.00 x precision)**, 3/5 negative	same, `pn-s5-e003.json`	5 paired cells
larger starting window, `n0` 2048 -> 8192	**−0.00063 ± 0.00135 (−0.08 x precision)**	`pn-s6-n0-8192.json` vs `pn-s1-grow.json`, paired	3 designs x 3 seeds

! `n0` IS ALSO NULL, and that overturns the expectation in section 6.2. Starting the window 4x larger does not close the gap — at a resolution 50x finer than the stage-1 effect it would have to explain. So the mechanism is NOT "the first 2048 events are memorised" in its simplest form, and a larger `n0` is not the campaign fix I expected it to be. Since the FULL window does close the gap (§2.1) while 4x does not, the effect is not a smooth function of the starting size: either it is the act of growing itself, or the epoch count that growing forces. `n0` = 32768 was still running.

Parameter noise has FAILED. D54 made stage 5 the discriminator: noise helps → path dependence, a cheap knob; noise fails but re-init works → the memorised early data is the cause; neither → the schedule is not the mechanism. The re-init arm (`pn-s4-reinit`) is queued and settles it.

`weight_decay` was the standing lead and has partly retracted itself: `output/screen/architecture-study.md` §3b measured `wd = 1e-1` costing +0.0063 of level and NARROWING the span 0.2546 → 0.2428, and withdrew its own "span widens, level improves" headline. What survives is a gap cut (4–5x on three designs), which is the failure mode of buying precision by underfitting both sides. `pn-s3-wd1e-3` and `pn-s3-wd1e-4` are queued, capped at 1.0e-3 by the user.

2.3 What `loss_precision` the task can actually deliver at m=4/r32

All from `output/screen/precision-arms-handover.md` unless noted. Every figure is a `diff + err`.

schedule	designs x seeds	max	mean	note
growth (arm A)	16 x 2 = 32 cells	**0.03359**	0.01203	max is a hand-built corner; over the 12 Sobol designs alone the max is **0.0183**
fixed window (arm C)	10 x 2 = 20 cells	**0.01304**	0.00517	all cells exit on the plateau test, so all are floors
growth, paired, `n_models=1` (stage 1)	3 x 3 = 9 cells	0.02178	0.01263	6 capped
fixed, paired, `n_models=1` (stage 1)	3 x 3 = 9 cells	0.00876	0.00600	9 converged

Pre-registration §5 requires the MAXIMUM over >= 8 designs x >= 2 seeds at the budget actually chosen. On the growth path that is 0.03359 (or 0.0183 if the four hand-built physics corners are excluded by declaration — they are inside the box and a search CAN propose them, so 0.03359 is the protocol number). On the fixed-window schedule the equivalent statistic is 0.01304.

What each implies for criterion (d) on the NEURAL span of 0.2619:

```
loss_precision             bar = 10x    % of neural span    seeds clearing at 20-
>40 (neural-scaled)
0.03359 growth max           0.3359           128%                0/10    <- pre-
registration 5.1 auto-REJECT
0.01304 fixed-window max   0.1304          50%             2/10
0.01263 paired grow mean   0.1263          48%             2/10
0.00882 p90 pairwise + err 0.0882          34%             7/10
0.00800 configured         0.0800          31%             8/10
0.00621 median pairwise+err 0.0621         24%             9/10
0.00600 paired fixed mean  0.0600          23%             9/10
```

So the candidate's fate under criterion (d) is decided entirely by which quantity the bar is built from, and that is a question about the criterion, not about the physics — still the user's call, and explicitly NOT to be changed because a candidate failed at 10x. The cancellation-aware reading was MEASURED (D44): within one seed, across designs at the good end (9 designs, 36 pairs, uncensored at the cap), `diff` has mean 0.00913, sd 0.00193, pairwise |Δ| median 0.00183 and p90 0.00444 — 5x below the bias level and 18x below the maximum. Criterion (d) compares `best@n` against `best@2n`, both good designs within one run carrying the same seed, so that pairwise spread IS its error term; the seed-level offset cancels exactly. `diff` does NOT track loss level once censoring is removed (Spearman −0.083 and +0.267, sign flips between seeds); the +0.709 seen earlier was a censoring artefact.

2.4 More read-outs make the gap worse

0.0104 max at the 8-read cell against 0.03359 at 32 reads — more read-outs give the network more to memorise. **!** The two numbers come from different design sets: 0.0104 is the max over the 2 designs that crashed the cost runs at r8, 0.03359 is the max over 16 designs x 2 seeds at r32 including hand-built corners. The DIRECTION is supported; the 3.2x magnitude is not, because a max over 32 cells is not comparable to a max over 2. Excluding corners the r32 figure is 0.0183. This trades against §1.2: readings widen the span (good) and coarsen the precision (bad), and on the proxy the span effect wins — bar/span falls 32.6% → 21.9% from r8 to r32 at a fixed bar.

2.5 The measurement horizon fits the reaction; the concentration box does not (mm-sym)

Measured on `config/detector/enzyme_mm_sym.yaml`, 2048 events per design, reported PER EXPERIMENT because `[A]0` is a design coordinate spanning three decades within one design.

Half-times run 0.25–2.0 h against a 1.0 h window — median 0.71 of the duration, 5–95% at 0.28–1.82 — so the window spans about half a half-life to four of them, and in every design measured, ZERO experiments were over before the first read-out. **The horizon is fine.**

What is not fine is `concentration_A_bounds: [0.01, 10.0]` mM against an **absolute** `measurement_noise` of 0.05 mM:

```
design (m=4)       median swing/noise  experiments that are pure noise  never reacted
sobol-best   0.1406           3.96                 1 of 4                        0
sobol-median 0.2440           4.32                 0 of 4                        1
sobol-worst  0.3335           1.09                 2 of 4                        2
m=32 uniform-random           2.96                 9 of 32                       7
m=32 centre of box            4.31                 0 of 32                       0
```

An experiment at [A]0 = 0.015 mM reads 3.2x its own [A]0 at the first sample — that is noise, not chemistry. Two consequences. The proxy's RANKING does track real information content (swing/noise falls monotonically best → median → worst), which is reassuring for the ordering even though §1.1 disputes the levels. And at m=32 a uniformly random design wastes a quarter to a half of its experiments, so a random baseline there will look terrible and BO will show a large gain — but part of that gain is only "avoid the dead bottom decade of the box", which under the NO TRICKS clause is a bound that encodes the answer. **If m=32 goes forward, the `[A]0` lower bound must be raised to something the read-out can resolve, and that must be declared BEFORE the campaign.**

3. The meta handicap

What is measured. Arm C (`ContinualTrainer`: one warm-started network, 50/50 replay) minus arm A (`DesignTrainer`, fresh network) at the SAME design, scored on the SAME 65536 held-out events disjoint from all training indices. The pool is RECONSTRUCTED exactly from a recorded BO trajectory — `spent` decomposes uniquely into `2048 + m*1024` train rows plus `683 + m*341` validation rows, checked on 11 designs. Script `scripts/probe_meta_cripple.py`.

! Every meta number in this project is on the OLD three-class `inhibitor` trajectory (`output/campaign-inhib/126382657/from_scratch/results.json`), not on `extremes`. Whether the handicap reproduces on the campaign task is UNMEASURED.

3.1 The handicap is real, and it is a bias

At k = 6 / 13 / 19 (not 5 / 10 / 20: iterations 20–21 of that trajectory sit at 0.998 / 0.993, i.e. AT the ceiling, where no arm can resolve anything), seed 7:

arm		k=6	k=13	k=19	k=19 in loss_precision
C - A	continual vs from_scratch	+0.0184	+0.0214	+0.0326	4.1 x
D - A	continual at 2x data	+0.0176	+0.0127	+0.0249	3.1 x
B - A	from_scratch at 2x data	-0.0015	+0.0026	+0.0013	0.2 x
E - A	XGBoost on C's own rows	+0.0184	+0.0763	+0.0839	10.5 x

- **It is a BIAS, not under-training.** Doubling every per-design count recovers 0 / 41 / 24% of it.
- **It is not data volume.** Doubling `from_scratch`'s own budget moves it by <= 0.003 (0.4x precision).
- **The network is not the limitation.** XGBoost on arm C's own rows is identical at k=6 and 0.055 / 0.051 WORSE at k=13 / 19. The deep set is the best estimator available on that pool.

The first multi-seed reading: three training seeds at k = 19, from `output/screen/meta-sweep-k19-s{7,11,23}.json` and `meta-capacity-k19-s{7,11,23}.json`.

probe	s7	s11	s23	mean	sd	x loss_precision
meta-sweep	+0.0360	+0.0290	+0.0321	+0.0324	0.0035	4.04
meta-capacity	+0.0438	+0.0273	+0.0302	+0.0338	0.0088	4.22

So the handicap at k=19 is **~4 x `loss_precision`**, reproduces across three seeds and two independent probe runs, and the across-seed sd is 11–26% of the mean.

What a single fit here can resolve. At the SAME seed and the same nominal settings, arm C's held-out loss at k=19 read 0.8291, 0.8385, 0.8291 and 0.8370 across four runs — a run-to-run spread of **0.0094 = 1.2 x `loss_precision`**. Arm A at k=6 read 0.8453 vs 0.8485 (0.0032). This is the denominator D53 requires; any effect below ~0.010 must be judged against it.

3.2 ! "The handicap GROWS with designs seen" is NOT established

D48 reported the growth 0.0184 → 0.0214 → 0.0326 across k and drew a large consequence from it (that D14 is an artefact with a drifting offset, which is what redefined this round's tuning target). The second seed does not show it:

C - A	k=6	k=13	k=19	
seed 7	+0.0154	+0.0214	+0.0419	rising
seed 11	+0.0194	+0.0141	+0.0194	flat

The seed-7 rise is 0.0142–0.0265 depending on which run is used, against a measured run-to-run spread of 0.0094 at a single k. **The SIGN of the handicap is solid** (positive in every cell of every run at both seeds and all three k). **The TREND is one seed.** Anything resting on the trend should be re-read as provisional.

3.3 REFUTED: dropout on the design channels

The structural premise is correct — `detopt/nn/set_regressor.py:188-196` puts `nnx.Dropout` before the first shared linear, and the last four feature channels of each experiment are its design coordinates, so at `p_dropout: 0.1` each design number is dropped independently 10% of the time. The predicted consequence does not follow. Measured at k=19 on all three seeds, design channels pushed 0.1 of the scaled range INWARD so corner-piled designs never clip:

quantity	arm A (from_scratch)	arm C (continual)
design_tv (prediction move)	0.0472 / 0.0486 / 0.0471	0.0854 / 0.0721 / 0.0894
blind_delta (loss when blind)	+0.0752 /+0.0837 /+0.0833	+0.1919/+0.1959/+0.2044
w(design)/w(measurement)	1.050 / 1.013 / 1.057	1.078 / 1.041 / 1.052

`meta` is roughly **2x more design-sensitive** and pays **2.4x more** to be blinded to the design. The shrinkage story predicts the opposite on both, on every seed. `from_scratch` is a null reference here, not a comparison: its design channels are constant within a design, so those weights are unconstrained rather than learned.

3.4 REFUTED: capacity — and the "~37% cut" note is a seed-7 artefact

A note reported mid-run that "2x parameters and 2x data each cut the handicap ~37% at seed 7". Checked against `output/screen/meta-capacity-k19-s{7,11,23}.json` (base 10444 parameters, wide 20876):

cell	s7	s11	s23	mean	cut vs base
base (1x params, 1x data)	+0.0438	+0.0273	+0.0302	+0.0338	—
2x parameters	+0.0274 (38%)	+0.0295 (-8%)	+0.0287 (5%)	+0.0285	16%
2x data	+0.0282 (36%)	+0.0266 (3%)	+0.0299 (1%)	+0.0283	16%
2x parameters AND 2x data	+0.0354 (19%)	+0.0326 (-19%)	+0.0264 (12%)	+0.0315	7%

CONFIRMED at seed 7 (38% and 36%); REFUTED across seeds. The mean cut is 16%, i.e. 0.0053 absolute = 0.66 x `loss_precision`, and the across-seed sd of the base cell alone (0.0088) is larger than it. The 2x-both cell is WORSE than either single doubling, which no capacity story explains. Capacity is not the mechanism and doubling it is not a fix.

3.5 Dropout level is a large modifier; `weight_decay` is the one lever that has moved it

From `output/screen/meta-sweep-k19-s{7,11,23}.json`, three seeds, paired within cell:

cell	A level (mean)	C level (mean)	C - A mean	sd	x precision
p=0.1 wd=1e-6	0.7922	0.8245	+0.0324	0.0035	4.04
p=0.1 wd=1e-2	0.7918	0.8136	+0.0218	0.0007	2.73
p=0.2 wd=1e-3	0.8421	0.9577	+0.1156	0.0207	14.45
p=0.3 wd=1e-2	0.8711	0.9987	+0.1276	0.0186	15.95
p=0.3 wd=1e-6	0.8720	1.0118	+0.1398	0.0081	17.47

! The p >= 0.2 rows are NOT usable as a dropout effect. Their C cells sit at 0.94–1.02, at or above the three-class ceiling of 1.0, and their per-segment logs show rounds ending at exactly the 64-epoch `min_epochs` floor with validation drift as bad as −0.039 (`output/screen/mc-sweep.log`), i.e. not converged. Those cells are confounded by under-training and by sitting at the no-information value.

What IS clean: `weight_decay` 1e-6 → 1e-2 at `p_dropout` 0.1 cuts the handicap by 0.0106 (1.3 x `loss_precision`), on all three seeds (−0.0146 / −0.0064 / −0.0107), and it does so by making `meta` BETTER (C level 0.8245 → 0.8136) while leaving `from_scratch` untouched (0.7922 → 0.7918). It is the only intervention measured so far that reduces the handicap without costing either arm. Two caveats: 1e-2 is an order of magnitude above the 1.0e-3 cap the user set for the precision ablation, and its effect on the LANDSCAPE SPAN is unmeasured — the melt evidence says strong `weight_decay` narrows the span, which would trade the handicap against the very quantity criteria (c) and (d) are measured on.

3.6 Replay: a live suspect, and a decision-log claim the second seed contradicts

`ContinualTrainer._sample_indices` gives the design under evaluation only HALF of every minibatch, the rest replayed from all past designs, while the number the trainer must drive under `loss_precision` and the number handed to BO are evaluated on the CURRENT design's window alone — a fixed 2x dilution from k=2 onward (`detopt/nn/trainer/continual.py:30-45, 62-91`).

```
C - A                      k=6      k=13     k=19
replay_weight 1.0  s7  +0.0154  +0.0214  +0.0419
replay_weight 0.1  s7  +0.0253  +0.0216  +0.0175
replay_weight 1.0  s11 +0.0194  +0.0141  +0.0194
replay_weight 0.1  s11 +0.0187  +0.0177  +0.0352
replay_weight 0.0  s11 +0.0105  +0.0127  +0.0047
```

! D52's conclusion — "downweighting replay helps ONLY when the pool is large", from the −58% at seed 7 / k=19 — is contradicted by seed 11, where the same cell is +82% WORSE. The sign of `C*(0.1) − C(1.0)` flips between seeds at every one of the three k. D52's mechanism story should be treated as withdrawn pending a third seed.

Replay OFF (`replay_weight = 0`) is different and is the most promising result in this section: at seed 11 it is lower than `replay_weight` 1.0 at all three k, and at k=19 it takes the handicap to +0.0047 = **0.6 x `loss_precision`**, i.e. essentially gone. ONE seed, one run each, so it is a lead, not a finding — but it isolates the replay MIXTURE rather than the warm start as the carrier.

3.7 What this means for the campaign

At the design counts the campaign will run, the handicap is a systematic of about **+0.032 (4 x `loss_precision`)** running AGAINST `meta`. Against the r32 20 → 40 median gain of 0.1584 proxy / **0.1134 neural-scaled**, an arm offset of 0.032 is ~28% of the effect `meta` has to show. Not fatal; the largest known systematic; and measured on a different task.

Two harness consequences follow and neither is fixed:

- `bo.py` compares arms by their REPORTED losses, which are `(train + val)/2` on each arm's own windows — two different estimators. The campaign must report each arm's achieved `diff + err` alongside its loss (pre-registration §4), or a level difference cannot be told from a resolution difference.
- `bo.py` HARD-FAILS the whole run when any design cannot reach `loss_precision`, deterministically per seed. Recording the shortfall and continuing would make an 8-hour campaign robust to a stochastic training outcome. RAISED, not made — it changes what a campaign measures.

The historical **+0.0103** offset quoted throughout the older documents traces to `output/capacity/probe.log` — the MELT task, 4 designs, 1 repeat set, median of 4 paired differences, `meta` worse on 4/4, at a spend ratio of 1.46x. Different task, different scale from the +0.018…+0.033 measured here; the two must not be quoted as the same number.

---

4. The convergence rule

What the trainer uses now. `convergence_rule = "slope"` (the default) and `require_stable_gap = False` (the default), i.e. the four-branch procedure documented at `detopt/nn/trainer/design.py:7-24`:

```
1. (val - train) - err > loss_precision  [AND, if require_stable_gap, the gap has stopped falling]
   -> add data, before convergence
2. else train and validation not both plateaued over `patience` -> train on
3. else |val - train| + err >= loss_precision -> add data
4. else -> return ((train+val)/2, diff+err)
```

`config/enzyme_extremes.yaml` sets neither knob, so the campaign as configured runs the ORIGINAL rule 1 — the one §2.1 shows firing at the first legal epoch of every round.

Alternatives tested and rejected:

candidate	result	evidence
gap-stability gate (`require_stable_gap`)	NULL on the gap (−3.0% on `diff`), +56% wall clock	1 capped-in-both cell
Bayesian rule (`convergence_rule="bayes"`)	terminates, does NOT help: `diff` 0.00849 vs 0.00712 (+19%), slack 0.01165 vs 0.01006, 875 s vs 423 s (2.07x), 3490 epochs vs 2159	1 cell, capped in both, like-for-like
"distance to asymptote", fixed basis `a + b/sqrt(t)`	**REJECTED — hangs.** On an exponential tail it never fires in 40000 epochs on 5/5 seeds. On its own basis it is 5x closer to the floor for 28x the epochs	D56, synthetic harness
shape-adaptive residual, `b + C*f(t)` with the shape by MLE and `(C,b)` Bayesian — offset `f = 1/sqrt(t+alpha)` and exponent `f = t^-alpha`	**REJECTED — moves the hang rather than removing it.** Offset hangs on an exponential tail (3/3 seeds); exponent hangs on `A/sqrt(t)` (3/3), the truth it CAN represent and whose shape it recovers (fitted alpha 0.44–0.46 against a true 0.5), because with the exponent free the coefficients stop being identified	D57, synthetic harness
fixed-window weights in the Bayesian fit	**never terminates**: effective sample size pinned, `sd(slope)` frozen at 3.09e-4, and on a genuinely converged loss the test reads P = 0.79 / 0.88 / 0.72 / 0.86 at 12 / 30 / 100 / 400 epochs	`detopt/utils/training.py:101-135`
geometric decay weights	fails identically (effective sample size capped at `1/(1-lambda)`)	same

Why every residual test fails and the slope test does not (D57). A residual test asks where the curve ENDS UP, which is an extrapolation to `t = infinity`; any misfit or weak identification leaves the extrapolated residual bounded away from zero forever, and no amount of data fixes it because the data never reach infinity. The slope test asks whether the curve is still MOVING, which is local, and a flattening curve always eventually answers it — it converged in 18/18 synthetic runs across three tail shapes and both patiences. Full table:

```
truth          slope pat=8      slope pat=32     offset            exponent
A/sqrt(t)      811 ep, 2.91x    2288 ep, 1.73x   31859 ep, 0.45x   HANG (3/3)
A/t            548 ep, 0.59x    1326 ep, 0.25x    3936 ep, 0.11x    2022 ep, 0.14x
A*exp(-t/40)  2377 ep, 0.26x    5337 ep, 0.00x   HANG (3/3)        29474 ep, 0.00x
```

```
(epochs to converge; residual = TRUE distance of train from its own limit when the test fires,
 in units of loss_precision = 8.0e-3)
```

What was kept: harmonic `1/k` weighting (newest = 1, previous = 1/2, …) in `bayesian_trend`. The weights sum like the harmonic series, so the effective sample size grows without bound

— 3.1 at n=12 to 7.3 at n=800 — `sd(slope)` collapses 3.6e-4 → 4e-6, and the test fires from n=30 on a converged loss while correctly holding off on a still-decaying one.

Two retractions worth keeping visible. The "63x" and "14955x" decay-rate overestimates reported for the Bayesian fit were `fitted / true_local`, a ratio whose denominator collapses as the curve flattens — it diverges by construction and says nothing about whether the rule works. And the "2.9x above its own limit when the slope rule fires" residual is a property of the SYNTHETIC `1/sqrt(t)` tail, not of the rule: the same rule leaves 0.59x on a `1/t` tail and 0.26x on an exponential one. Whether the real curves decay as slowly as `1/sqrt(t)` is NOT established.

The honest knob is the HORIZON, not the threshold: `patience` 8 → 32 cuts the residual 2.91 → 1.73 x `loss_precision` for 2.8x the epochs, while raising the probability threshold 0.9 → 0.999 buys 3%. That measurement is why `patience` is now 32 in the defaults (D58).

On the liveness bound. D50 and D51 record a `waited_long_enough` bound being added to the `require_stable_gap` gate and to the Bayesian branch. It is not in the code, and that is correct: the user ruled *"Rule liveness should not exist"* and it was removed from both branches. The decision log records the addition and not the removal; this note is the correction. The plain rule always terminates regardless, because `iteration_limit` is enforced on every addition.

---

5. Open questions, and what is still running

Running / queued (2026-08-13 21:00; two GPU shards, so the ablation runs two at a time):

```
578  pn-s6-n0     n0 in {8192, 32768} at precision 8.0e-3  <- removes the §2.1 confound
579  pn-s7-sched   warmup/patience 16/8 vs 4/32 on the real task
582/583  pn-s2-grow / pn-s2-fixed   dropout OFF, grow and fixed
584      pn-s5-e03                   parameter noise eps = 0.03
585     pn-s4-reinit                re-initialise the network at every data addition
586/587  pn-s3-wd1e-4 / wd1e-3      weight_decay swept, capped at 1.0e-3 (user)
593    mm-neural             mm-sym best/median/worst through the SetRegressor (CPU)
600     mm-neural-m32            mm-sym at m=32, designs named by construction (CPU)
```

Every ablation arm is PINNED to `--warmup-epochs 16 --patience 8` so it stays differenceable against stage 1, even though the defaults have since changed (D58).

! Stage 2 is the exception and cannot be differenced against stage 1. `nnx.Dropout` is only CONSTRUCTED when the rate is > 0, so it consumes rng state; `--p-dropout 0` therefore yields a DIFFERENT initial network from the same seed. Stage 2's two arms (grow and fixed, both at dropout 0) are paired against each other and are valid as a pair; neither may be compared to a stage-1 arm.

! Six ablation jobs were once started by the scheduler beyond shard capacity and died immediately on `CUDA_ERROR_NO_DEVICE`, having consumed their arm while exiting "complete". `stage.sh` now checks `CUDA_VISIBLE_DEVICES` and requeues itself (bounded by `SLURM_RESTART_COUNT`) rather than run an arm on a CPU fallback.

! Some `output/screen/pn-s{2,3,4,5-e03,6,7}*.log` files on disk are from an earlier attempt that died immediately at `resolve_device` ("Unknown backend cuda") because it was submitted without a GPU shard. They contain no results; do not read them as arms.

Open, in the order they bite:

1. **Is `mm-sym`'s rejection a proxy artefact?** Jobs 593 and 600. If the neural floor is near zero the candidate comes back and the whole Stage A verdict is re-opened. §1.1.
2. **Which quantity criterion (d)'s bar is built from.** At the growth-path maximum the bar is 128% of the neural span, which pre-registration §5.1 makes an automatic reject; on any cancellation-aware reading it is 23–34%. Changing the 10x multiple because a candidate failed at 10x is the move NO TRICKS bans; the question is whether the criterion as written measures what it intends to.
3. **Does the growth-vs-fixed effect survive at equal training length?** Stage 6 answers it. The grow arm's mean final window was 102969 against the fixed arm's 131072 —

growth saved 21% of the samples and paid 63x the epochs for it, so a larger `n0` may be cheaper in BOTH currencies.

4. **Is the schedule the mechanism at all?** Parameter noise has failed, so D54's stage-5 discriminator hinges on `pn-s4-reinit`.
5. **Does the `meta` handicap exist on `extremes`?** Every measurement is on the three-class inhibitor trajectory. Nothing has been reconstructed on the campaign task.
6. **Is `replay_weight = 0` the fix?** One seed, three k, all favourable, k=19 at 0.6 x precision. Needs a second and third seed. It changes what `meta` IS — a warm start without replay is a different strategy, and that is a decision, not a bug fix.
7. **Condition (iv), dead volume as a proxy for "too easy".** Dead volume is collapsing (12.1 → 5.7 → 1.0% across r8, r32, m6) while the span WIDENS, which is the opposite of the failure the condition was written to catch. Not changed, because moving a pre-registered condition after results exist is banned; flagged so the m=6 exclusion is read as a possible proxy artefact.
8. **Post-hoc budget truncation is NOT verified** (pre-registration §7.1 PRECONDITION). `bo.py` terminates on pool exhaustion and the pools are ring buffers, which is favourable, but `iteration_limit` sets `steps_per_epoch` AND the eval kernel's size, so "a run to 40 cut at 20" has not been shown to equal "a run to 20".
9. **Neural per-design cost at the CHOSEN cell is UNMEASURED.** §6.3.

---

6. The settings the campaign should be launched at

`config/enzyme_extremes.yaml` + `config/detector/enzyme_extremes.yaml`, as they stand, with the provenance of every load-bearing value. **MEASURED** = there is an artefact on disk for this task; **INHERITED** = carried from another task or an earlier round and never re-measured here; **UNMEASURED** = no measurement exists at this operating point.

6.1 Detector — `config/detector/enzyme_extremes.yaml`

key	value	status	source
`mechanism_classes`	`[mostly_competitive, mostly_uncompetitive]`	declared	`output/screen/provenance-extremes.md`; the only line differing from `enzyme_inhib.yaml`
`n_experiments`	4 (design dim 16)	structural	one experiment per cell of the 2x2 the physics implies; user: "m = 4 is enough"
`n_measurements`	**32**	**MEASURED**	widest span of the three completed cells (0.3659), largest 20→40 gain (+0.1584), lowest bar/span (21.9%) — §1.2
`n_steps_per_measurement`	**40** (1280 TOTAL)	**MEASURED**	campaign-scale guard scan at 2e6 events, worst corner: 8.83x margin at 1280 against 1.62x at the shipped 640. Total held constant across the readings sweep

key	value	status	source
integration_tolerance	5.0e-3	rule	10% of measurement_noise; left at the stricter 10% because the guard has never fired
measurement_noise	0.05	INHERITED, frozen	baseline, set once before any landscape; NO TRICKS forbids moving it now
box bounds, priors, n_stages, damping	as shipped	declared	provenance-extremes.md; every bound states its rule from the prior alone

6.2 Training — config/enzyme_extremes.yaml

key	value	status	source / what to do
loss_precision	8.0e-3	**INHERITED — the blocking measurement**	from addendum-inhibitor-precision.md (three-class chemistry), never measured on the binary task. Growth path at this cell delivers a maximum of 0.03359 (0.0183 excluding hand-built corners); the fixed-window schedule delivers 0.01304. Pre-registration §5 requires the MAXIMUM over >= 8 designs x >= 2 seeds at the chosen budget, fixed BEFORE the verdict. **Do not launch until this is set from §2.3.**
n0	2048	**! the lever under test**	6 of 9 paired cells CAPPED from n0 = 2048; every fixed-window cell converged. pn-s6-n0 measures 8192 and 32768 at precision 8.0e-3. Expect this line to change
n_increment	1024	INHERITED	the doubled schedule (2048) is a measured NULL on both gap and cost

key	value	status	source / what to do
`iteration_limit`	131072	INHERITED	sets `steps_per_epoch` = 512 AND the eval kernel size AND the per-design window cap; measured on the three-class task
`budget`	2097152	**! INHERITED, and probably too large**	§6.3
`warmup_epochs`	**4**	user's ruling (D58), **UNMEASURED on the real task**	small on purpose: the settled-test is a posterior, so an unsettled early curve fails it through its own width. Every precision arm on disk ran 16, which is why they are pinned to 16
`patience`	**32**	user's ruling (D58), synthetic evidence only	8 → 32 cuts the residual 2.91 → 1.73 x precision for 2.8x the epochs (§4). Every precision arm on disk ran 8. `pn-s7-sched` measures 16/8 against 4/32 on the real task
`flatness_tol`	1.0	INHERITED	`is_plateaued` fires when the fitted drop over `patience` epochs is below `flatness_tol * loss_precision`
`convergence_rule`	(unset → `"slope"`)	**MEASURED**, keep	bayes is +19% on `diff` at 2.07x wall clock; both residual rules hang (§4)
`require_stable_gap`	(unset → `False`)	**MEASURED**, keep off	NULL on 1 cell, +56% wall clock
`batch` / `eval_batch` / `val_fraction`	256 / 2048 / 0.25	INHERITED	unchanged all round
`optimizer.adamw.learning_rate`	2.5e-4	INHERITED	LR-schedule study found constant/SGDR/cyclical converge at identical data and epochs

key	value	status	source / what to do
optimizer.adamw.weight_decay	1.0e-6	INHERITED	1e-1 cuts the gap 4–5x but costs +0.0063 of level and NARROWS the span. pn-s3-wd1e-{3,4} queued, capped at 1.0e-3 by the user. Separately, 1e-2 cuts the meta handicap by 1.3 x precision on 3/3 seeds (§3.5) — untested for its span cost
regressor.set-regressor.features	[[24,16],[16,24]]	INHERITED	10444 parameters at n_models = 4, 3162 at 1. Doubling to [[34,24],[24,34]] does not fix the meta handicap (§3.4)
regressor.set-regressor.n_models	4	INHERITED	! the on-task ensemble probe ran at a design scoring 0.995 against a ceiling of 1.0 and cannot resolve anything; its 4-vs-8-vs-16 numbers are not evidence
regressor.set-regressor.p_dropout	0.1	INHERITED (melt)	! same problem: the dropout probe ran at a 0.995-loss design. The config's "costs 0.0067 of level, buys a 5.3x cheaper exit" is a MELT-task measurement. pn-s2 measures dropout off on this task
plot_per_epoch	false	**MEASURED**	~2.4 min of the 396 s on one measured design, 36% of wall clock, all after convergence
bo.n_init	5	required	the user's criterion is stated for "BO with 5 initial Sobol points"
bo.gp.kernel	normalised-invariant-rbf, exchangeable: 4	**MEASURED**	the batch is a SET; the invariant kernel beats ard-rbf at p = 0.0003 once its silently-NaN gradient was fixed

6.3 Campaign size — measure before launching

Measured on the extremes cost runs (output/cost-extremes{,-meta,-s2}/results.json), 7 from_scratch designs and 7 meta designs:

```
from_scratch  78391 calls/design over 7 completed designs -- a FLOOR: the 2 designs that crashed
                      were the expensive ones and each burned the full 174763-
call cap. Counting them at
              the cap gives 99807. Per-design spread 5.2x in calls, 6.3x in seconds
meta           31981 calls/design over 7 designs (run completed)
ratio          78391 / 31981 = 2.45x        <- the design advantage, MEASURED
rate           261-343 calls/s, stable to +-10% design to design
```

At `budget = 2097152` that is ~21–27 designs for `from_scratch` and ~66 for `meta` — both sides of the 20 → 40 pair r32 passes. Wall clock is the problem: 1.7–2.2 h per run x 10 runs (5 seeds x 2 arms) / 2 shards = **8.5–11 h against the ~8 h target**, at the operating point the cost runs used.

! The cost runs were made at 640 TOTAL integration steps; the campaign cell runs 1280. The per-call detector work roughly doubles. So the figures above are a LOWER bound and the budget almost certainly needs cutting to ~1.5e6 calls. Pre-registration §7 requires this be MEASURED with a short `bo.py` run at the chosen cell, not projected. Do that first.

6.4 What must happen before the launch

1. Resolve `mm-sym` (jobs 593/600). A near-zero neural floor re-opens the task choice.
2. Measure `loss_precision` at m=4/r32 under the schedule the campaign will actually run (stage 6 decides whether that is grown-from-`n0`-8192/32768 or the current 2048), over >= 8 designs x >= 2 seeds, take the MAXIMUM, and FIX it in writing. It is the one value the verdict hangs on.
3. Reconcile `warmup_epochs` / `patience` with whatever `pn-s7-sched` returns.
4. Measure per-design cost at the chosen cell and size `budget` to fit ~8 h for 5 seeds x 2 arms on 2 shards.
5. Get the user's ruling on criterion (d)'s bar (§2.3) and on condition (iv) (§5 item 7).
6. Launch once, long, and cut post factum at the declared points, with `meta` cut at the same DETECTOR-CALL budget as `from_scratch`, never the same design count (`scripts/campaign_criterion.py --compare-arms-at`). Report all three pairs whatever they say.

---

7. Claims that are NOT supported by anything on disk

- **"the 3-class task is bit-identical (verified: 0.9343 before and after)"** — `docs/HANDOFF.md:76`, asserting that adding `mechanism_classes` to the shared detector left the three-class task unchanged. **UNVERIFIED.** The only artefacts containing 0.9343 in a relevant position are melt-task kernel-symmetry runs in a worktree and have nothing to do with the three-class inhibitor. No before/after pair for that change exists in `output/`. The claim may well be true — the config diff is genuinely one line — but it has not been demonstrated and must not be repeated as measured.
- **"`meta` reads +0.0103 higher than `from_scratch` at identical designs"** — repeated in four task plans and twice in the pre-registration as though it were a property of the arms. It traces to `output/capacity/probe.log`: the MELT task, 4 designs, one repeat set, median of 4 paired differences, at a spend ratio of 1.46x. Not an `extremes` number, not an inhibitor number, and not on the cross-entropy scale the campaign uses.
- **"2x parameters and 2x data each cut the handicap ~37%"** — true at seed 7 (38% / 36%), false at seeds 11 and 23 (−8% / +5% and +3% / +1%). §3.4.
- **"downweighting replay helps when the pool is large" (D52)** — contradicted by seed 11, where the same cell is 82% worse. §3.6.
- **"the `meta` handicap GROWS with designs seen" (D48)** — one seed. Flat at seed 11. §3.2.
- **`p_dropout: 0.1` and `n_models: 4`** — both rest on on-task probes run at a design scoring 0.995 against a ceiling of 1.0, which by the project's own rule ("state what a probe must separate and check it can") cannot resolve anything. They are inherited defaults, not measurements.